Experiment 11: Travelling Salesperson Problem (TSP) Using Exhaustive Search

Aim

To write a Python program to solve the Travelling Salesperson Problem (TSP) using the exhaustive search approach and analyze its time complexity.

Algorithm

1. Start.
2. Read the number of cities and the distance matrix representing distances between cities.
3. Fix the first city as the starting city.
4. Generate all possible permutations of the remaining cities.
5. Calculate the total travel distance for each possible route, including the return to the starting city.
6. Find the route with the minimum total distance.
7. Display the optimal route and its minimum distance.
8. Stop.

Source Code

```
from itertools import permutations

def tsp(distance, n):
    cities = list(range(1, n))
    min_cost = float('inf')
    best_route = []

    for route in permutations(cities):
        current_route = (0,) + route + (0,)
        cost = 0

        for i in range(len(current_route) - 1):
            cost += distance[current_route[i]][current_route[i + 1]]
```

```
        if cost < min_cost:
            min_cost = cost
            best_route = current_route

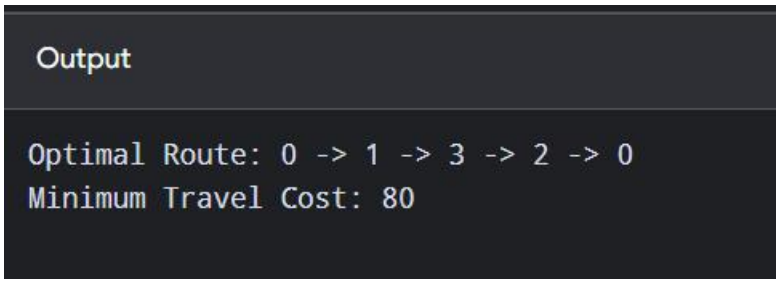
    return best_route, min_cost


distance = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]

n = len(distance)
route, cost = tsp(distance, n)

print("Optimal Route:", " -> ".join(map(str, route)))
print("Minimum Travel Cost:", cost)
```

Output :



Result

Thus, the Python program was successfully executed, and the Travelling Salesperson Problem was solved using the exhaustive search approach. The optimal route was 0 → 1 → 3 → 2 → 0, with a minimum travel cost of 80 units.